Testiranje Angular aplikacije



Autor: Goran Savić

Angular testiranje

- Potrebno je izvršiti jedinično testiranje Angular klijentske aplikacije
- To podrazumeva testiranje svih komponenata u klijentskoj aplikaciji
- Testovi treba da izolovano testiraju svaku komponentu
- Primeri
  - `angular-students/*.*.spec.ts`



Angular testiranje

- Angular sadrži *Testing utilities*
  - Skup klasa i funkcija koje olakšavaju testiranje
  - Omogućuju da testirana komponenta komunicira sa Angular radnim okvirom
  - Test objekti se testiraju u Angular okruženju
- Neke testove je moguće realizovati i kao potpuno izolovane testove nezavisno od Angulara
  - ručno se instanciraju objekti, umesto da se injektuju od Angulara

TestBed

- Klasa koja omogućuje da se test izvrši u Angular okruženju
  - čini dostupnim objekte čijim životnim ciklusom upravlja Angular
- *configureTestingModule* metoda
  - uspostavlja inicijalno test okruženje
  - pri inicijalizaciji se šalje TestModuleMetadata objekat koji predstavlja modul koji će test okruženje sadržati

TestBed

- *inject* metoda
  - preuzima objekat iz okruženja
  - uvek treba preuzimati objekat iz okruženja
    - umesto ručnog instanciranja
    - jer druge komponente u okruženju koriste objekat iz okruženja, pa je samo korišćenjem tog objekta moguće imati ispravan test
- *createComponent* metoda
  - kreira Angular komponentu unutar okruženja
    - u skladu sa konfiguracijom okruženja
  - vraća objekat klase ComponentFixture

ComponentFixture

- omogućuje pristup Angular komponenti i njenoj DOM reprezentaciji
- *componentInstance* atribut
  - instanca same komponente na koju se objekat odnosi
- *debugElement* atribut
  - objekat klase DebugElement koja predstavlja reprezentaciju elementa DOM stabla za potrebe testiranja

ComponentFixture

- *detectChanges* metoda
  - inicira poziv Angular ciklusa koji detektuje promene u objektima i sinhronizuje ih sa HTML komponentama
  - ako test postavi vrednost u objekat, pre provere da li se ta vrednost nalazi u HTML komponenti, potrebno je pozvati *detectChanges*
  - koristi se i za inicijalizaciju komponente jer automatski poziva *ngOnInit*

ComponentFixture

- *whenStable* metoda
  - vraća *Promise* objekat koji se razrešava kada je *ComponentFixture* stabilan
    - tj. kada je Angular ciklus detekcije promena završen uključujući završetak svih asinhronih aktivnosti u ciklusu
  - koristi se da test sačeka da se promene detektuju i postave u komponente

DebugElement

- Omogućuje testu pristup elementima HTML DOM stabla
- *query* metoda
  - omogućuje preuzimanje jednog HTML elementa po nekom selektoru
  - **`debugElement.query(By.css('#fieldX'));`**
- *queryAll* metoda
  - slično kao *query*, samo što vraća sve elemente koji odgovaraju selektoru
- *nativeElement* atribut
  - predstavlja sam HTML element u *browseru*

Asinhroni pozivi - waitForAsync

- waitForAsync funkcija omogućuje jednostavnije testiranje funkcionalnosti koje se izvršavaju asinhrono
- kod unutar waitForAsync funkcije se izvršava unutar specijalne asinhrone test zone
- ova funkcija presreće i evidentira sve *Observable* objekte kreirane unutar nje
  - koristi se za detekciju kada je *ComponentFixture* stabilan pozivom *whenStable* metode

Asinhroni pozivi - fakeAsync

- Dodatno pojednostavljenje u odnosu na *async* funkciju za testiranje asinhronog koda
- omogućuje linearno pisanje koda, bez reakcije na razrešavanje *Observable* objekta putem *subscribe* metode
- *tick* funkcija
  - unutar *fakeAsync* funkcije možemo simulirati protok vremena dok god se sve trenutno nerešene asinhrone aktivnosti ne završe
- nakon poziva funkcije, asinhrone aktivnosti su završene, pa ne mora kod da bude u funkciji *subscribe* na *Observable* objektu

Testiranje HTTP komunikacije

- Standardno se vrši postavljanjem test dvojnika na mesto komponente koja komunicira sa backendom
- HttpTestingController se koristi kao dvojnik
  - ne vrši stvarne mrežne pozive
  - možemo pratiti i simulirati mrežne pozive

Praćenje HTTP poziva

- Proveravamo da li je poziv ka tom URL upućen i dobijamo mock objekat za simuliranje odgovora

```
const mockHttp =
    httpTestingController.expectOne(
        'api/students');

expect(mockHttp.request.method).
      toBe('GET');
```

Simuliranje HTTP odgovora

- Vrši se metodom flush i specificiranjem lažnog odgovora

```
const mockResponse = {
      id: 1,
      cardNumber: 'a123',
      firstName: 'Petar',
      lastName: 'Petrovic'
 };

mockHttp.flush(mockResponse);
```

Primer testiranja servisa

- angular-students
  - student.service.spec.ts

Test dvojnici

- Za izolovano testiranje komponente potrebno je postaviti test dvojnike na mesto objekata koje komponenta koristi
- Možemo koristiti Jasmine podršku za spy objekte
  - omogućuju vraćanje predefinisanih izlaza na predefinisane ulaze
  - omogućuju „špijuniranje“ da li su i kako objekti korišćeni
- Za komplikovanije test dvojnike, možemo napraviti poseban objekat koji simulira stvarno ponašanje

Test dvojnici - primer

```
let studServiceMock = {
  getStudents: jasmine.createSpy('getStudents')
   .and.returnValue(of({body:
      [{}, {}, {}]}))
};
...
TestBed.configureTestingModule({
 declarations: [ StudentsComponent ],
 providers: [
 {provide: StudentService, useValue: studServiceMock },
 { provide: Router, useValue: routerMock } ]
});
...
expect(studentService.getStudents).toHaveBeenCalled();
```

Komunikacija sa HTML komponentama

- Da bi se inicirao *binding* podataka od objekta ka komponenti
  - detectChanges metoda iz klase ComponentFixture
- Da bi se inicirao *binding* od komponente ka povezanom objektu
  - *dispatchEvent metoda* nad *nativeElement* objektom u testu

DispatchEvent

```
function newEvent(eventName: string,
    bubbles = false, cancelable = false) {
  let evt =
    document.createEvent('CustomEvent');
  evt.initCustomEvent(eventName, bubbles,
    cancelable, null);
  return evt;
}
...
let cardNumberInput =
    fixture.debugElement.query(
       By.css('#field1c')).nativeElement;
cardNumberInput.dispatchEvent(newEvent('input'));
```

Primer testiranja komponente

- angular-students
  - students.component.spec.ts
  - student-detail.component.spec.ts